

Algoritmi 2

*I seguenti appunti sono a cura di **Catalin Ceban** e si basano sul corso di "Algoritmi 2" erogato nell'A.A. 2026 dal professore Paul Joseph Wollan per il C.d.L Informatica (33503)*

27 settembre 2026

Indice

1	Teoria dei Grafi	2
1.1	Il problema dei ponti di Königsberg	3
1.2	Rappresentazione fisica dei grafi	6
1.2.1	Matrici di adiacenza	6
1.2.2	Liste di adiacenza	7
2	Passeggiate e cammini	9
2.1	Algoritmo di ricerca di un ciclo all'interno di un grafo	10

Capitolo 1

Teoria dei Grafi

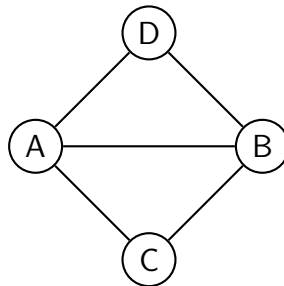
Definizione 1.0.1: Grafo

Un **grafo** è una coppia $G = (V, E)$, dove:

- V rappresenta l'insieme dei **vertici** (o nodi).
- E rappresenta l'insieme degli **archi** (o lati). Matematicamente parlando, un arco è a sua volta una coppia di valori (v_1, v_2) , dove v_1, v_2 sono dei vertici del grafo.

Per comodità di notazione, nel corso del manuale denoteremo con $n = |V|$ il numero dei vertici e con $m = |E|$ il numero degli archi.

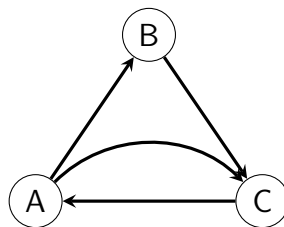
A titolo esemplificativo, consideriamo la seguente rappresentazione grafica di un grafo:



Riguardo agli archi, potremmo avere anche un orientamento.

Definizione 1.0.2: Grafo diretto (o orientato)

Un grafo si dice **diretto** o **orientato** se ciascuno dei suoi archi ha un orientamento, cioè punta specificatamente verso un dato nodo.



È importante notare che possiamo avere anche due archi sugli stessi nodi ma con orientamento diverso (come mostrato nel collegamento bidirezionale).

Definizione 1.0.3: Arco incidente e vertici adiacenti

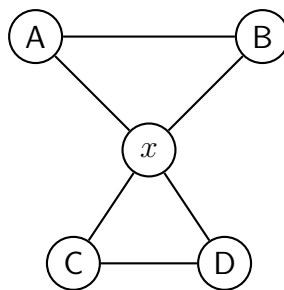
Sia f un arco che collega due vertici x e y , formalmente rappresentabile come $x \leftarrow f \rightarrow y$. Diciamo che:

- L'arco f è **incidente** ai vertici x e y .
- Il vertice x è **adiacente** al vertice y .

Definizione 1.0.4: Grado di un vertice

Il **grado** di un vertice x , denotato con $\deg(x)$, è dato dal numero di archi incidenti in x .

Ad esempio, considerando un nodo centrale collegato a quattro vertici periferici:



Per il nodo centrale x contrassegnato, avremo chiaramente $\deg(x) = 4$, poiché vi incidono esattamente quattro archi **distinti**.

1.1 Il problema dei ponti di Königsberg

Il problema classico da cui nasce la teoria dei grafi è il famoso problema dei ponti di Königsberg:

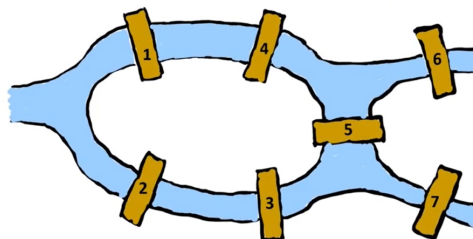


Figura 1.1: Rappresentazione grafica del problema dei ponti di Königsberg.

Ci si chiede: *è possibile attraversare tutti i ponti una sola volta e tornare al punto di partenza?* Come prima cosa, quando si vuole tradurre un problema in un problema di teoria dei grafi, è buona cosa cercare di dare una rappresentazione grafica il più accurata possibile del problema. Essendo questo un problema di piccole dimensioni, possiamo ottenere la seguente struttura:

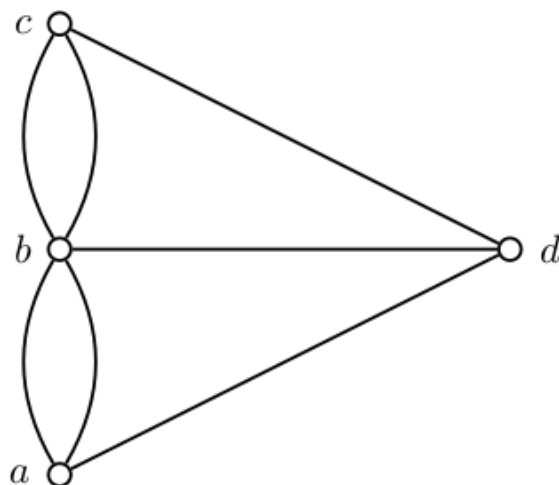


Figura 1.2: Rappresentazione a grafo del problema dei ponti.

Si invita il lettore a provare manualmente se esiste tale sequenza di attraversamenti, ma la risposta alla domanda rimane **negativa**. Prima di giustificarne la motivazione, dobbiamo fornire delle importantissime definizioni riguardanti i grafi.

Definizione 1.1.1: Passeggiata in un grafo

Una **passeggiata** (o **walk**) in un grafo è una sequenza $v_0, e_1, v_1, e_2, v_2, \dots, v_{k-1}, e_k, v_k$ che alterna vertici e archi. Formalmente, può essere anche indicata con $W = (v_1, v_2, \dots, v_k)$, dunque omettendo gli archi ma indicando solo in ordine di apparizione i vertici.

Definizione 1.1.2: Passeggiata chiusa

Una passeggiata si dice **chiusa** se $v_0 = v_k$, cioè il punto di inizio coincide con il punto di arrivo.

Possiamo adesso giustificare, più o meno, la risposta al problema dei ponti di Königsberg: qualsiasi grafo con almeno un nodo x con grado dispari **non** ammette una passeggiata chiusa che utilizza tutti gli archi una sola volta.

E se invece tutti i nodi avessero grado pari? Interessante, domanda, ma **non è detto**.

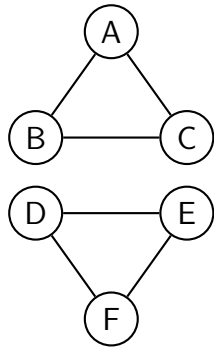


Figura 1.3: Grafo non connesso formato da due componenti triangolari disgiunte. Tutti i vertici hanno $\deg(v) = 2$, ma il grafo complessivo non è connesso e dunque non è possibile effettuare una passeggiata chiusa al suo interno.

Definizione 1.1.3: Grafo connesso

Un grafo si dice **connesso** se per ogni coppia di vertici esiste una passeggiata da x a y . In poche parole, un grafo si dice connesso se ogni coppia di vertici scelta è collegata in qualche modo.

Teorema 1.1.1: Caratterizzazione di Eulero

Esiste una passeggiata chiusa che contiene ogni arco una sola volta se e solo se:

1. Ogni vertice ha **grado pari**.
2. Il grafo è **connesso**.

1.2 Rappresentazione fisica dei grafi

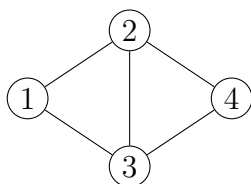
La rappresentazione informatica dei grafi in memoria è fondamentale per determinare l'efficienza degli algoritmi che operano su di essi. Esistono principalmente due modalità standard per rappresentare un grafo: le **matrici di adiacenza** e le **liste di adiacenza**.

1.2.1 Matrici di adiacenza

Si stabilisce una matrice $n \times n$ dove $n = \#V$, e dove:

$$m_{i,j} = \begin{cases} 1 & \text{se } i \text{ è adiacente a } j \\ 0 & \text{altrimenti} \end{cases}$$

Esempio:



$$\begin{array}{c} \begin{matrix} & 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

Se il grafo non è diretto, la matrice sarà simmetrica rispetto alla diagonale.

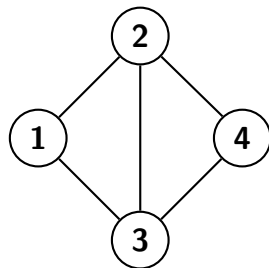
Vediamo di seguito il **costo computazionale** di alcune delle operazioni elementari che possono essere effettuate sui grafi:

- **Visita del grafo e memorizzazione:** $\Theta(n^2)$, in quanto viene memorizzato come una matrice $n \times n$.
- **Per verificare se i è adiacente a j :** $\Theta(1)$, in quanto è sufficiente controllare il valore di $M[i][j]$: se esso corrisponde ad 1 allora i due nodi sono adiacenti, altrimenti no.
- **Verificare il grado di un dato vertice i :** $\Theta(n)$, in quanto occorre scorrere la riga i -esima della matrice e contare quanti 1 ci sono.

1.2.2 Liste di adiacenza

Una lista di adiacenza è composta da:

- Un array in cui ogni elemento è un nodo del grafo.
- Per ogni nodo del grafo, dunque per ogni elemento dell'array, viene associato anche un puntatore **neigh** che punta ad una lista concatenata contenente tutti i nodi adiacenti al nodo corrente.



$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$	$\rightarrow [2, 3]$
$\begin{bmatrix} 2 \\ 3 \end{bmatrix}$	$\rightarrow [1, 3, 4]$
$\begin{bmatrix} 3 \\ 4 \end{bmatrix}$	$\rightarrow [1, 2, 4]$
$\begin{bmatrix} 4 \end{bmatrix}$	$\rightarrow [2, 3]$

Costo computazionale delle operazioni elementari:

- Per visitare e memorizzare: $\mathcal{O}(n + \sum_v \deg(v)) = \mathcal{O}(n + 2m) = \mathcal{O}(n + m)$.
Spieghiamo il perché di questo costo computazionale:
 1. Sicuramente, per memorizzare una lista di adiacenza abbiamo bisogno innanzitutto di un array che tenga traccia di tutti i nodi. Ecco da dove viene il $\mathcal{O}(n)$.
 2. Inoltre, per ogni nodo dobbiamo memorizzare anche tutti i nodi a lui adiacenti. Il numero di nodi adiacenti corrisponde al **grado** del singolo nodo, ossia al numero di archi incidenti al nodo. Sappiamo che il numero di archi è quantificabile con m , ma in questo caso stiamo contando ogni arco ben due volte (vedendo anche l'esempio, nel nodo 1 stiamo dicendo che è adiacente a 2, e nel nodo due stiamo dicendo che è adiacente ad 1). Dovremo dunque contare due volte ogni arco, ottenendo una quantità pari a $2m$. Il costo computazionale diventa dunque nell'ordine di $n + 2m$, ma sapendo che le costanti moltiplicative sono indifferenti asintoticamente, tale valore diventa semplicemente $\mathcal{O}(n + m)$.
- Per verificare se i è adiacente a j : $\mathcal{O}(\deg(i))$. Questo in quanto come vediamo la lista di adiacenza relativa ad ogni nodo ha un numero di elementi pari al grado di quel nodo. Dunque per verificare se due nodi sono adiacenti o meno è necessario scorrere la lista di adiacenza di quel nodo e cercare il nodo adiacente interessato.

Per quanto riguarda la memorizzazione e la visita completa di un grafo memorizzato mediante liste di adiacenza, si potrebbe fare un appunto riguardo ai grafi diretti. In tal caso, a meno che ogni arco non sia bi-direzionale, verrebbe memorizzata letteralmente **la metà** dei nodi memorizzati in un grafo non orientato.

Osservazione 1.2.1

Supponiamo di avere un problema che ci richieda frequentemente quali nodi puntano verso un dato nodo u . Se mantenessimo un'implementazione mediante liste di adiacenza come quella vista in figura, per trovare tutti i nodi collegati ad u dovremmo scorrere per ogni nodo nella lista le sue liste di adiacenza, ottenendo un costo computazionale di $\mathcal{O}(n + m)$, come se fosse una normale visita.

Se invece decidessimo di modificare leggermente la struttura della lista di adiacenza, aggiungendo per ogni nodo un puntatore ai suoi nodi entranti e uscenti, il costo computazionale di tale operazione si ridurrebbe ad un misero $\mathcal{O}(\deg(u))$, in quando dovremo scorrere solo la lista dei nodi entranti (il cui numero corrisponde appunto al grado del nodo). Nota bene che questa implementazione non appesantisce così tanto il costo di memorizzazione della struttura dati, in quanto per ogni nodo u si aggiunge $\mathcal{O}(\deg(u))$ di costo computazionale, portando il totale ad $\mathcal{O}(n + 3m = \mathcal{O}(n + m))$.

Capitolo 2

Passeggiate e cammini

Definizione 2.0.1: Cammino

Un **cammino** nel grafo è una passeggiata in cui non viene ripetuto alcun vertice.

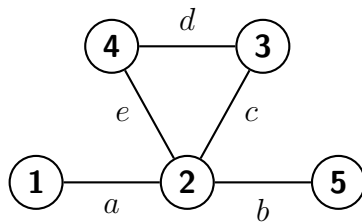


Figura 2.1: Una passeggiata potrebbe essere: 1 a 2 c 3 d 4 e 2 b 5.

Una camminata da 1 a 5 potrebbe invece essere 1 a 2 b 5.

Corollario 2.0.1

Esiste una passeggiata da x a y $\iff$ esiste un cammino da x a y .

Dimostrazione 2.0.1

Chiaramente, se esiste un cammino allora sicuramente esiste anche una passeggiata, dato che ogni cammino è un particolare tipo di passeggiata. Abbiamo così dimostrato la prima implicazione ($\Rightarrow$).

Viceversa, supponiamo di avere una passeggiata $W = (v_0, v_1, \dots, v_K)$. Abbiamo due possibilità:

1. Se tutti i vertici sono distinti, allora la passeggiata è già un cammino.
2. Se invece abbiamo almeno un indice ripetuto, cioè esistono i, j tali che $v_i = v_j$ con $i < j$.

Questo significa che all'interno della passeggiata è presente un sottociclo:

$$W_1 = (v_i, v_{i+1}, \dots, v_{j-1}, v_j)$$

ma sapendo che $v_i = v_j$, la struttura si chiude. Possiamo eliminare W_1 dalla passeggiata iniziale mantenendo però il nodo in cui parte la diramazione (in

questo caso v_i). La passeggiata iniziale può essere dunque riscritta come:

$$W = (v_1, \dots, v_i, \underbrace{v_{i+1}, \dots, v_{j-1}, v_j}_{W_1}, v_{j+1}, \dots, v_K)$$

Eliminando W_1 otteniamo un cammino più corto, e reiterando il procedimento rimuoviamo tutti i cicli ottenendo infine un cammino semplice.

Definizione 2.0.2: Ciclo

Un **ciclo** è un grafo connesso in cui **ogni vertice ha grado 2**.

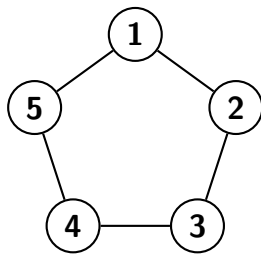
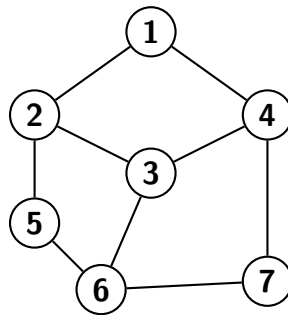


Figura 2.2: Grafo ciclo a 5 vertici (C_5).

2.1 Algoritmo di ricerca di un ciclo all'interno di un grafo

Proponiamo un altro problema classico del lavoro con i grafi: supponiamo che, dato un grafo G con l'ipotesi che **ogni vertice ha grado 2 o superiore**, volessimo trovare un qualsiasi ciclo nel grafo. Partiamo con un esempio grafico di un grafo che contiene un ciclo, e vediamo come procederemmo umanamente.



L'idea di algoritmo chiaramente consiste nel:

1. Scegliere un nodo qualsiasi di partenza.
2. Memorizzarlo all'interno di un apposita struttura dati.

3. Prendere un arco e andare al nodo adiacente. **Importante** è assicurarsi che il prossimo nodo non sia ancora stato visitato. In tal caso, significa che abbiamo fatto un giro e abbiamo trovato un ciclo.
4. Altrimenti, tornare all'inizio.

Messa un'idea di base, procediamo con la scrittura dell'effettivo algoritmo in pseudocodice e in seguito alla sua spiegazione.

```
1  def trovaCiclo(G):
2      L = lista vuota
3      x = un vertice qualsiasi del grafo
4      L.append(x)
5
6      current = x
7      next = x.neighb[0]
8
9      while next not in L:
10         L.append(next)
11
12         if next.neighb[0] == current:
13             u = next.neighb[1]
14         else:
15             u = next.neighb[0]
16
17         current = next
18         next = u
19
20     while L[0] != next:
21         L.remove(0)
22
23     return L
```

Listing 2.1: Pseudocodice per la ricerca di un ciclo all'interno di un grafo.

Spiegazione algoritmo 2.1.1: Estrazione di un ciclo da un grafo

1. Definiamo una lista vuota, per tenere traccia dei nodi visitati finora.
2. Prendiamo un qualsiasi vertice dal quale iniziare la nostra ricerca.
3. Aggiungiamo il nodo scelto all'interno della lista.
4. Prendiamo come prossimo nodo il primo della lista degli adiacenti al nodo corrente.

5. Eseguiamo un ciclo che va avanti finché il `next` che abbiamo preso non è un nodo già visitato: in tal caso significa che abbiamo intrapreso una strada circolare e siamo tornati indietro.
6. All'interno del ciclo aggiungiamo alla lista `L` dei nodi visitati anche `next`.
7. A questo punto dobbiamo scegliere il nuovo `next`, ma dobbiamo assicurarci che non sia il nodo da cui siamo partiti: ecco perché controlliamo che `next.neighbor[0]` sia diverso dal nodo su cui siamo adesso, in modo da non poter tornare *subito* indietro.
8. Aggiorniamo i puntatori dei nodi per la prossima iterazione.
9. Una volta usciti da questo ciclo, significa che abbiamo intrapreso la strada circolare di cui si parlava prima, e che dunque abbiamo trovato un ciclo. Significa inoltre che ora il puntatore `next` punterà al nodo da cui inizia la diramazione del ciclo, e dunque sarà il primo di questo. Ecco perché, nel ciclo finale, controlliamo che `L[0] != next`, perché `next` è il primo nodo del ciclo, e dunque deve essere anche il primo della lista che dovremo restituire.

Bisogna fare un appunto su questo algoritmo: ora come ora, per come l'abbiamo scritto, il suo costo computazionale è un $\mathcal{O}(n^2)$, tutto per colpa del controllo di testa del primo ciclo `while`. La condizione del ciclo è un `next not in L`, che richiede tempo $\mathcal{O}(n)$ in un array disordinato. Dato che il ciclo `while` potrebbe poi richiedere fino ad un massimo di n iterazioni, una per ogni nodo del grafo, il costo computazionale sale al tempo quadratico. Possiamo ovviamente ottimizzare questo codice, presentando la seguente versione:

```
1  def trovaCiclo(G):
2      L = lista vuota
3      vis = [0] * n
4
5      x = un vertice qualsiasi del grafo
6      L.append(x)
7      vis[x] = 1
8
9      current = x
10     next = x.neighbor[0]
11
12     while vis[next] == 0:
13         L.append(next)
14         vis[next] = 1
15
16     if next.neighbor[0] == current:
```

```

17         u = next.neighbr[1]
18     else:
19         u = next.neighbr[0]
20
21     current = next
22     next = u
23
24     while L[0] != next:
25         L.remove(0)
26
27     return L

```

Listing 2.2: Ricerca di un ciclo in un grafo ottimizzata

Spiegazione algoritmo 2.1.2: Ricerca di un ciclo in un grafo ottimizzata

Andremo a commentare solo le nuove scelte implementative: Si è scelto di aggiungere un array `vis` che verrà usato per tenere traccia degli elementi visitati. Ciò viene fatto stabilendo una biezione tra l'identificativo del nodo e il suo indice all'interno dell'array `vis`. Così facendo il controllo in testa al ciclo `while` diventa semplicemente un controllo accedendo ad un elemento dell'array tramite indice, dunque eseguibile in tempo costante. L'aggiunta di un array `vis` che ha una grandezza massima dell'ordine di del numero di nodi n aumenta il costo spaziale da $\Theta(n)$ a $\Theta(n + n) = \Theta(n)$, dunque rimane nello stesso ordine asintotico.

Possiamo ora discutere riguardo al costo computazionale dell'algoritmo, per entrambe le versioni:

- Versione inefficiente:
 - Costo computazionale: $\mathcal{O}(n^2)$
 - Costo spaziale: $\Theta(n)$
- Versione efficiente:
 - Costo computazionale: $\Theta(n)$
 - Costo spaziale: $\Theta(n)$